Κώδικας 14.1 *Παράδειγμα χρήσης Iterator με ArrayList.*

Στην main του κώδικα 14.1 καλείται καταρχάς η loadData που φορτώνει στο ArrayList, store, τα δεδομένα 7 προσώπων. Η κλάση ArrayList διαθέτει τη μέθοδο iterator που επιστρέφει έναν Iterator για πρόσωπα. Καλώντας την iterator λαμβάνουμε έναν τέτοιον Iterator και τον τοποθετούμε στην αναφορά it. Διαμέσου αυτής της αναφοράς προσπελαύνουμε ένα-ένα τα στοιχεία της ArrayList με την it.next(). Αυτή θα πρέπει να καλείται μόνον εφόσον έχουμε ελέγξει πως υπάρχει διαθέσιμο επόμενο στοιχείο, δηλαδή μόνον εφόσον η it.hasNext() επιστρέψει true. Να σημειωθεί πως σε αυτό το παράδειγμα, η it.next() επιστρέφει ένα αντικείμενο τύπου Person.

Ας δούμε και ένα παράδειγμα που θα βοηθήσει να κατανοήσουμε τις δυνατότητες πολυμορφικού κώδικα που παρέχονται μέσω των διεπαφών.

```
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;

public class PolymorphismDemo {

      public static List<Person> loadData() {
        List<Person> rVal = new ArrayList<>();
        rVal.add(new Person("10", "Jim", "Smith", LocalDate.of(1961,
Month.JANUARY, 1)));
        rVal.add(new Person("20", "Adam", "Smith", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("1", "John", "Black", LocalDate.of(1971,
Month.JANUARY, 1)));
        rVal.add(new Person("2", "John", "Dalton", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("12", "Bill", "Adams", LocalDate.of(1965,
Month.JANUARY, 5)));
        rVal.add(new Person("6", "Averell", "Dalton", LocalDate.of(1972,
Month.JANUARY, 1)));
        rVal.add(new Person("7", "John", "Cyan", LocalDate.of(1980,
Month.JANUARY, 1)));
        return rVal;
    }

    public static void main(String[] args) {
        List<Person> store = loadData();
        for (int i=0; i<2; i++) {
            Iterator<Person> it = store.iterator();
            while (it.hasNext()) {
                System.out.println(it.next());
            }
            System.out.println("-------------------------------------");
            store = new LinkedList<>(store);
        }
    }
}
```

Κώδικας 14.2 *Προσπέλαση των στοιχείων μιας ArrayList και μιας LinkedList με τον ίδιο κώδικα*

Στον κώδικα 14.2 η loadData έχει τροποποιηθεί ώστε ο τύπος επιστροφής της δεν είναι ArrayList<Person> όπως στον κώδικα 14.1 αλλά List<Person>. Η List είναι προκαθορισμένη διεπαφή. Τόσο η κλάση ArrayList όσο και η κλάση LinkedList υλοποιούν τη διεπαφή List. Αυτό σημαίνει πως μια ArrayList είναι μια List και μια LinkedList είναι επίσης μία List. Επομένως, η loadData που η τυπική τιμή επιστροφής της είναι List<Person> μπορεί να επιστρέψει πραγματικό αντικείμενο τύπου ArrayList<Person> ή LinkedList<Person> ή οποιασδήποτε άλλης συλλογής που υλοποιεί την List.

Στην main ορίζουμε επίσης τη μεταβλητή store ως List<Person>. Επομένως στη μεταβλητή αυτή μπορεί να εκχωρηθεί αναφορά σε πραγματικό αντικείμενο οποιασδήποτε κλάσης υλοποιεί την List. Αν μεταβούμε στην τεκμηρίωση της List θα διαπιστώσουμε ότι ανάμεσα σε άλλες μεθόδους ορίζει και μία μέθοδο Iterator<E> iterator(). Κάθε σαφής κλάση που υλοποιεί τη διεπαφή List εγγυάται τη διαθεσιμότητα αυτής της μεθόδου. Με άλλα λόγια σε σχέση με το παράδειγμά μας, τόσο η ArrayList όσο και η LinkedList διαθέτουν μέθοδο Iterator<E> iterator().

Στην main επομένως εκχωρούμε στην store την αναφορά του ArrayList που επέστρεψε η LoadData. Στη συνέχεια, όπως ακριβώς και στον κώδικα 14.1 λαμβάνουμε τον Iterator δια μέσου του οποίου προσπελαύνουμε τα στοιχεία του ArrayList. Όμως, τον κώδικα που προσπελαύνει τα στοιχεία της λίστας τον επαναλαμβάνουμε 2 φορές. Τη δεύτερη φορά έχουμε αλλάξει τη λίστα μας από ArrayList<Person> σε LinkedList<Person>. Ωστόσο ο κώδικας τρέχει σωστά και για τις δύο διαφορετικές δομές, καθώς και οι δύο είναι τύπου List και άρα και οι δύο παρέχουν Iterator. Να σημειωθεί ότι ο Iterator που παρέχεται σε κάθε περίπτωση βασίζεται σε μια κλάση που υλοποιεί τη διεπαφή Iterator. Η ίδια η κλάση όμως είναι διαφορετική για την ArrayList και διαφορετική για την LinkedList.

14.1.2 Comparator

Μερικές φορές έχουμε την ανάγκη να ταξινομήσουμε τα αντικείμενα μιας συλλογής κατά ποικίλους τρόπους. Για παράδειγμα, έστω ότι θέλουμε να εμφανίσουμε τα πρόσωπα της λίστας του κώδικα 14.2 καταρχάς ταξινομημένα κατά id και στη συνέχεια ταξινομημένα κατά επώνυμο και όνομα. Σε αυτές τις περιπτώσεις είναι χρήσιμη η διεπαφή Comparator<T>. Η διεπαφή δηλώνει μια αφηρημένη μέθοδο ως εξής:

```
int compare(T o1, T o2)
```

Συγκρίνει τις δύο παραμέτρους της. Επιστρέφει θετικό ακέραιο αν θεωρεί το o1 μεγαλύτερο από το o2, αρνητικό αν θεωρεί το o1 μικρότερο από το o2 και 0 αν τα θεωρεί ίσα.

Ας δούμε ένα παράδειγμα:

```
import gr.ihu.cs.lmous.OOJ.k12.v1.x.Person;
import java.util.Comparator;
import java.util.Iterator;
import java.util.List;

public class ComparatorDemo {

    private static class CompareById implements Comparator<Person> {

        @Override
        public int compare(Person o1, Person o2) {
            int id1 = Integer.decode(o1.getId());
            int id2 = Integer.decode(o2.getId());

            return id1 - id2;

        }
    }

    private static class CompareByName implements Comparator<Person> {

        @Override
        public int compare(Person o1, Person o2) {
            String sName1 = o1.getsName();
            String sName2 = o2.getsName();
            String fName1 = o1.getfName();
            String fName2 = o2.getfName();
            int rVal = sName1.compareTo(sName2);
            if (rVal == 0) {
                return fName1.compareTo(fName2);
```

```
                }
                return rVal;
            }
        }

        static void prt(Iterator<Person> it) {
            while (it.hasNext()) {
                System.out.println(it.next());
            }
        }

        public static void main(String[] args) {
            List<Person> store = PolymorphismDemo.loadData();
            store.sort(new CompareById());
            prt(store.iterator());
            System.out.println("---------------------");
            store.sort(new CompareByName());
            prt(store.iterator());
        }
    }
```

Κώδικας 14.3 *Χρήση Comparator για ταξινόμηση λίστας*

Στην main φορτώνουμε καταρχάς τα πρόσωπα του κώδικα 14.2 στην store. Στη συνέχεια καλούμε την sort. Η συνάρτηση είναι υπερφορτωμένη. Έχουμε ήδη χρησιμοποιήσει την έκδοση που δεν έχει παραμέτρους. Εδώ χρησιμοποιούμε την έκδοση που απαιτεί ένα Comparator ως παράμετρο. Αρχικά περνάμε ένα αντικείμενο της εσωτερικής κλάσης CompareById. Η CompareById υλοποιεί τη διεπαφή Comparator<Person>, δηλαδή τα αντικείμενά της είναι Comparators για αντικείμενα Person. Επομένως, η CompareById πρέπει να παρέχει υλοποίηση της μοναδικής αφηρημένης μεθόδου της διεπαφής, δηλαδή της compare. Έτσι η sort μπορεί με ασφάλεια να καλέσει από τον Comparator την compare ώστε να μπορέσει να ταξινομήσει τα αντικείμενα Person κατά id.

Στη συνέχεια, η prt λαμβάνει ως παράμετρο τον Iterator της store μέσω του οποίου προσπελαύνει και εμφανίζει ένα προς ένα τα αντικείμενα της store.

Αμέσως μετά ταξινομείται η store με τον CompareByName οπότε η επόμενη κλήση της prt εμφανίζει τα πρόσωπα ταξινομημένα κατά επώνυμο και όνομα. Προσέξτε την εσωτερική κλάση CompareByName. Καταρχάς συγκρίνει δύο πρόσωπα κατά επώνυμο. Αν βρεθούν τα επώνυμα ίδια, τότε συγκρίνει κατά μικρό όνομα.

14.2 Υλοποίηση διεπαφής

Σε αυτήν την ενότητα παρουσιάζουμε ένα παράδειγμα δήλωσης και υλοποίησης μιας διεπαφής. Πιο συγκεκριμένα δηλώνουμε μια διεπαφή που προσομοιώνει τον Iterator της Java αλλά είναι εξειδικευμένη ειδικά για συλλογές του τύπου String. Αυτή η επιλογή γίνεται για καθαρά εκπαιδευτικούς λόγους καθώς μια τέτοια διεπαφή ορθότερα θα ήταν παραμετροποιημένος τύπος. Παράλληλα με τη διεπαφή παρουσιάζεται και μια κλάση που αξιοποιεί τη διεπαφή. Ας ξεκινήσουμε με την παρουσίαση της διεπαφής:

```
public interface MyIterator {

    public boolean hasNext();
    public String next();
}
```

Κώδικας 14.4 *Η διεπαφή MyIterator*

Όπως φαίνεται στον κώδικα 14.4, μια διεπαφή ορίζεται παρόμοια με μία κλάση μόνο που χρησιμοποιείται η λέξη-κλειδί interface αντί της class. Παράλληλα οι μέθοδοι της διεπαφής είναι αφηρημένες.
Ο κώδικας που ακολουθεί παρουσιάζει την εφαρμογή της MyIterator:

```
public class ArrayListOfStrings {
```

```
private String[] store = new String[10];
private int index = 0;

private class ArrayListIterator implements MyIterator {

    private int current = 0;

    @Override
    public boolean hasNext() {
        return current < size();
    }

    @Override
    public String next() {
        return get(current++);
    }
}

private String[] increment(String[] store) {
    String[] rVal = new String[store.length + 10];
    System.arraycopy(store, 0, rVal, 0, store.length);
    return rVal;
}

public void add(String s) {
    if (index == store.length) {
        store = increment(store);
    }
    store[index++] = s;
}

public String remove(int idx) {
    if (idx > index || idx < 0) {
        return null;
    }
    String rVal = store[idx];
    for (int i = idx; i < index; i++) {
        store[i] = store[i + 1];
    }
    index--;
    return rVal;
}

public String removeLast() {
    return remove(index - 1);
}

public String get(int idx) {
    if (idx >= index || idx < 0) {
        throw new IndexOutOfBoundsException();
    }
    return store[idx];
}

public boolean isEmpty() {
    return index == 0;
}

public int size() {
    return index;
```

```
    }

    public MyIterator myIterator() {
        return new ArrayListIterator();
    }

    public static void main(String[] args) {
        ArrayListOfStrings list = new ArrayListOfStrings();
        list.add("A");
        list.add("B");
        list.add("C");
        list.add("D");
        MyIterator it = list.myIterator();
        while (it.hasNext()) {
            System.out.print(it.next());
        }
        System.out.println();
    }

}
```

Κώδικας 14.5 *Υλοποίηση MyIterator και ArrayListOfStrings*

Καταρχάς, ας μελετήσουμε την κλάση ArrayListOfStrings. Πρόκειται για μια απλή συλλογή. Ως αποθηκευτικό χώρο χρησιμοποιεί τον πίνακα store που αρχικά δημιουργείται με μήκος 10. Η index δείχνει στην πρώτη θέση του πίνακα που είναι ελεύθερη και μπορεί να αποθηκευτεί μία νέα συμβολοσειρά. Κατά τη δημιουργία αντικειμένου ArrayListOfStrings, η index είναι ίση με 0 που σημαίνει ότι ο πίνακας είναι άδειος. Έτσι η μέθοδος isEmpty επιστρέφει true μόνο εφόσον η index είναι ιση με 0. Η add ελέγχει αν υπάρχει χώρος στον store. Αν όχι καλεί την increment και αυξάνει τη χωρητικότητα του store κατά 10. Η remove διαγράφει το στοιχείο στη θέση idx από τον εσωτερικό πίνακα μετακινώντας όλα τα στοιχεία από τη θέση idx και μετά μία θέση πριν την τρέχουσα θέση τους. Αυτό βέβαια μπορεί να γίνει εφόσον κληθεί με παράμετρο μεγαλύτερη από το 0 και μικρότερη από την index. Ενημερώνει κατάλληλα την index και επιστρέφει το στοιχείο που διαγράφηκε. Η get επιστρέφει το στοιχείο στη θέση idx και η size τον αριθμό των συμβολοσειρών της ArrayListOfString.

Ας πάμε τώρα στην myIterator. Η τυπική τιμή επιστροφής της είναι MyIterator. Έτσι επιστρέφει ένα αντικείμενο της κλάσης ArrayListIterator που υλοποιεί τη διεπαφή MyIterator. Η ArrayListIterator διατηρεί τον δείκτη current που καθορίζει ποιο είναι για το στιγμιότυπο τύπου MyIterator το τρέχον στοιχείο της ArrayListOfString. Κατά τη δημιουργία των αντικειμένων ArrayListIterator, ο δείκτης current αρχικοποιείται στο 0. Έτσι κάθε νέο αντικείμενο ArrayListIterator θεωρεί ως πρώτο στοιχείο αυτό που βρίσκεται στη θέση 0 του store. Η hasNext επιστρέφει true εφόσον ο current είναι μικρότερος από την τιμή που επιστρέφει η size. Η δε next() επιστρέφει το στοιχείο στη θέση current του store και αυξάνει τον current κατά 1.

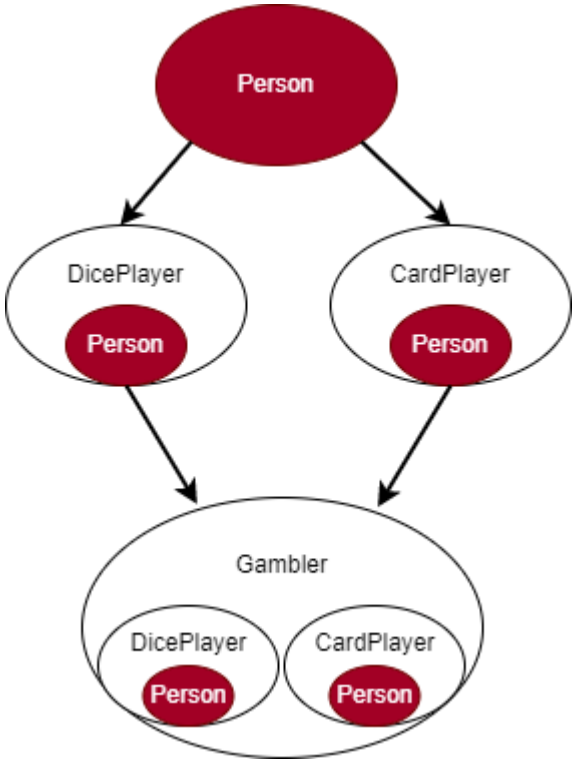
Η έξοδος της main είναι ABCD.

Ας σημειωθεί πως η ArrayListIterator θα μπορούσε να υλοποιεί την προκαθορισμένη διεπαφή Iterator της Java. Χρησιμοποιήθηκε εδώ η MyIterator ως μια ευκαιρία για να επιδειχθεί πως δηλώνουμε μια δική μας διεπαφή.

14.3 Πολλαπλή κληρονομικότητα

Ας υποθέσουμε πως έχουμε μια κλάση που αναπαριστά παίκτες ζαριών, έστω την DicePlayer, και μία κλάση που αναπαριστά χαρτοπαίκτες, έστω την CardPlayer. Έστω ότι χρειαζόμαστε και μία κλάση τα αντικείμενα της οποίας είναι ταυτοχρόνως και χαρτοπαίκτες και παίκτες ζαριού, έστω Gambler. Θα ήταν ίσως βολική η δυνατότητα η κλάση Gambler να κληρονομεί και την CardPlayer και την DicePlayer, δηλαδή η Gambler να έχει 2 γονικές κλάσεις. Εύκολα μπορούμε να σκεφτούμε παραδείγματα που ενδεχομένως θα ήταν χρήσιμο μια κλάση να έχει περισσότερες ακόμη και από 2 γονικές. Η σχέση κατά την οποία μία κλάση διαθέτει περισσότερες από μία γονικές είναι γνωστή ως πολλαπλή κληρονομικότητα (multiple inheritance).

Η πολλαπλή κληρονομικότητα μεταξύ κλάσεων χαρακτηρίζεται από το πρόβλημα του διαμαντιού (the diamond problem). Ανακαλέστε από την ενότητα 13.1 πως κάθε αντικείμενο παράγωγης κλάσης περιλαμβάνει ένα αντικείμενο της γονικής.



Σχήμα 14.1 *Το πρόβλημα του διαμαντιού*

Προσέξτε τώρα στο σχήμα 14.1 τι συμβαίνει αν η κλάση DicePlayer και η CardPlayer είναι παράγωγες της Person. Ένα αντικείμενο Person εσωκλείεται σε κάθε στιγμιότυπο της DicePlayer και της CardPlayer. Έτσι κάθε αντικείμενο της Gambler καταλήγει να περιλαμβάνει δύο αντικείμενα της Person. Αυτό βέβαια είναι μη αποδεκτό, καθώς αυξάνει ανώφελα τον όγκο των αντικειμένων Gambler και θέτει το ακόλουθο ερώτημα: Τι θα πρέπει να συμβεί αν αντικείμενο της Gambler καλέσει μια μέθοδο που αρχικά ορίζεται στην Person, π.χ. την getfName. Προσέξτε πως η getfName κληρονομείται και στην DicePlayer και στην CardPlayer. Επίσης λάβετε υπόψη πως η μέθοδος μπορεί να επανοριστεί ή όχι τόσο στην DicePlayer όσο και στην CardPlayer. Ποια getfName θα κληρονομήσει η Gambler; Επιπλέον, οι μεταβλητές στιγμιότυπου της Parent έχουν δημιουργηθεί και λάβει τιμές τόσο κατά τη δημιουργία του αντικειμένου DicePlayer όσο και κατά τη δημιουργία του CardPlayer. Ποιες μεταβλητές θα κληρονομήσει η Gambler; Αν υποθέσουμε πως η Gambler κληρονομεί τις μεταβλητές της Parent που δημιουργήθηκαν κατά την κατασκευή του αντικειμένου DicePlayer, οι μέθοδοι που κληρονομήθηκαν από την CardPlayer θα μπορούν να επενεργούν σε αυτές τις μεταβλητές;

Οι διάφορες σύγχρονες γλώσσες έχουν δώσει διαφορετική απάντηση στο πρόβλημα του διαμαντιού. Ας αποσαφηνίσουμε όμως κάποιες έννοιες σχετικά με την κληρονομικότητα ώστε να διευκολυνθούμε για να κατανοήσουμε τη λύση της Java.

Η κληρονομικότητα μπορεί να διακριθεί σε τρεις τύπους [2], την κληρονομικότητα κατάστασης (state inheritance), την κληρονομικότητα υλοποίησης (implementation inheritance) και την κληρονομικότητα τύπου (type inheritance). Η κληρονομικότητα κατάστασης αφορά τις μεταβλητές που κληρονομούνται από τη γονική, η κληρονομικότητα υλοποίησης τις υλοποιήσεις των μεθόδων που κληρονομούνται από τη γονική και τέλος η κληρονομικότητα τύπων αφορά τους τύπους που κληρονομούνται από τη γονική. Η κληρονομικότητα μεταξύ των κλάσεων περιλαμβάνει και τους τρεις τύπους κληρονομικότητας. Το προβλήματος του διαμαντιού όμως προκύπτει αποκλειστικά σε σχέση με την κληρονομικότητα κατάστασης και υλοποίησης. Επιπλέον, η κληρονομικότητα κατάστασης και υλοποίησης δεν είναι αναγκαίες προϋποθέσεις για την ανάπτυξη πολυμορφικού κώδικα. Αντίθετα η κληρονομικότητα τύπου είναι αναγκαία προϋπόθεση για την υποστήριξη πολυμορφικού κώδικα. Η Java δεν υποστηρίζει πολλαπλή κληρονομικότητα κατάστασης και υλοποίησης. Έτσι αντιμετωπίζει το πρόβλημα του διαμαντιού. Υποστηρίζει όμως πολλαπλή κληρονομικότητα τύπου δια μέσου των διεπαφών. Πράγματι οι διεπαφές διαθέτουν μόνο αφηρημένες μεθόδους και δεν επιτρέπουν τη δήλωση μεταβλητών στιγμιότυπου. Μία κλάση της Java μπορεί να κληρονομήσει μόνο από μία κλάση αλλά μπορεί να υλοποιήσει απεριόριστο αριθμό διεπαφών. Τα αντικείμενα της κλάσης συνδέονται με τη σχέση “είναι ένα” με κάθε διεπαφή που η κλάση υλοποιεί.

Ας δούμε τώρα πώς μπορούμε να αναπτύξουμε την κλάση Gambler στο πλαίσιο της Java έτσι ώστε κάθε αντικείμενο Gambler να είναι ταυτοχρόνως Person και DicePlayer και CardPlayer. Θα χρειαστούμε καταρχάς μια βοηθητική κλάση που αναπαριστά το ζάρι και δύο διεπαφές.

```
import gr.ihu.cs.lmous.OOJ.k11.v1.x.Lib;
import java.util.Arrays;

public class Dice {
    private final double[] distribution;

    public Dice(double[] distribution) {

        if (distribution.length != 6
                || !Lib.approximateEquals(Lib.sum(distribution), 1)) {
            throw new RuntimeException();
        }
        this.distribution = distribution;
    }

    public Dice() {
        distribution=new double[6];
        Arrays.fill(distribution, 1/6d);
    }

    public int roll() {
        double value = Lib.gen.nextDouble();
        double sum = 0;

        for (int i = 0; i < distribution.length; i++) {
            sum += distribution[i];
            if (value <= sum) {
                return i + 1;
            }
        }
        return 0;
    }
}

public interface DicePlayerI {

    public int roll(Dice dice);
}

import java.util.ArrayList;

public interface CardPlayerI {

    public Card pickTop(Deck deck);

    public ArrayList<Card> openCards();
}
```

Κώδικας 14.6 *Η κλάση Dice και οι διεπαφές DicePlayerI και CardPlayerI*

Σημειώστε πως η κλάση Dice και οι διεπαφές DicePlayerI και CardPlayerI πρέπει να τοποθετηθούν σε διαφορετικά αρχεία. Έχοντας στη διάθεσή μας τις δύο αυτές διεπαφές και την κλάση Dice, μπορούμε να ορίσουμε την κλάση Gambler που παρουσιάζεται στον κώδικα 14.7. Σημειώστε πως εδώ η κατανομή των πιθανοτήτων για το αποτέλεσμα του ζαριού δεν είναι πλέον χαρακτηριστικό του παίκτη αλλά του ζαριού που χρησιμοποιεί:

```
import java.time.LocalDate;
```

```
import java.util.ArrayList;

public class Gambler extends Person implements CardPlayerI, DicePlayerI {

    ArrayList<Card> cardsInHand = new ArrayList<>();

    public Gambler(String id, String fName, String sName, LocalDate
birthday) {
        super(id, fName, sName, birthday);
    }

    @Override
    public Card pickTop(Deck deck) {
        Card card = deck.pickCard();
        cardsInHand.add(card);
        return card;
    }

    @Override
    public int roll(Dice dice) {
        return dice.roll();
    }

    @Override
    public ArrayList<Card> openCards() {
        ArrayList<Card> rVal = new ArrayList<>(cardsInHand);
        cardsInHand.clear();
        return rVal;
    }
}
```

Κώδικας 14.7 *Η κλάση Gambler*

Η κλάση Gambler στον κώδικα 14.7 κληρονομεί από την Person (κώδικας 12.19) άρα διαθέτει τους τύπους, τις υλοποιήσεις και τις μεταβλητές της Person. Κάθε αντικείμενο Gambler επομένως είναι ένα Person. Επιπλέον υλοποιεί τις διεπαφές CardPlayerI και DicePlayerI. Αυτό σημαίνει πως για να είναι σαφής και όχι αφηρημένη, η Gambler πρέπει να παρέχει υλοποιήσεις για τις αφηρημένες μεθόδους των διεπαφών. Επομένως, κάθε αντικείμενο Gambler μπορεί να επιτελέσει τις λειτουργίες που ορίζονται στις CardPlayerI και DicePlayerI. Συνεπώς, κάθε αντικείμενο Gambler θεωρείται πως είναι ένας παίκτης ζαριών και ένας χαρτοπαίκτης. Ο κώδικας 14.8 επιδεικνύει τη χρήση της Gambler.

```
import java.time.LocalDate;
import java.time.Month;

public class GamblerDemo {

    public static void main(String[] args) {
        Deck deck = new Deck();
        double[] fairDist = new double[]{1d / 6, 1d / 6, 1d / 6, 1d / 6, 1d
/ 6, 1d / 6,};
        Dice dice = new Dice(fairDist);

        Gambler jim = new Gambler("01", "Jim", "Black", LocalDate.of(1976,
Month.JULY, 5));
        Gambler jack = new Gambler("02", "Jack", "Green",
LocalDate.of(1981, Month.APRIL, 12));
        deck.shuffle();
        for (int i = 0; i < 4; i++) {
            jim.pickTop(deck);
            jack.pickTop(deck);
        }
```

```
            System.out.println("Οι κάρτες του " + jim.getsName() + " " +
    jim.openCards());
            System.out.println("Οι κάρτες του " + jack.getsName() + " " +
    jack.openCards());
            System.out.println("Ζαριά του " + jim.getsName() + " " +
    jim.roll(dice));
            System.out.println("Ζαριά του " + jack.getsName() + " " +
    jack.roll(dice));
            CardPlayerI jimCP = jim;
            DicePlayerI jackDP = jack;
            Person jimP = jim;
        }
    }
```

Κώδικας 14.8 *Κάθε αντικείμενο της κλάσης Gambler είναι Person και DicePlayerI και CardPlayerI*

Εδώ ορίζουμε δύο αντικείμενα τύπου Gambler. Στη συνέχεια ανακατεύουμε μια τράπουλα και τους δίνουμε να τραβήξουν από τέσσερις κάρτες ο κάθε ένας. Μετά ανοίγουν τις κάρτες τους τις οποίες εμφανίζουμε συνοδευόμενες από το επώνυμο του αντίστοιχου παίκτη. Επομένως, μέχρι εδώ είναι σαφές πως κάθε αντικείμενο Gambler είναι ένα πρόσωπο και ένας χαρτοπαίκτης ταυτοχρόνως. Στο επόμενο βήμα τους βάζουμε να ρίξουν από μία ζαριά. Ας σημειωθεί πως η κλήση Dice dice = new Dice(fairDist) θα μπορούσε να αντικατασταθεί με την κλήση Dice dice=new Dice() με ίδιο αποτέλεσμα.

Οι τελευταίες τρεις γραμμές της main έχουν ως σκοπό να επιδείξουν πως αυτά τα αντικείμενα μπορούμε να τα δούμε μέσα στον κώδικά μας αποκλειστικά είτε σαν Person είτε σαν DicePlayer είτε σαν CardPlayer.

14.4 Άλλα χαρακτηριστικά

14.4.1 Κληρονομικότητα διεπαφών

Οι διεπαφές δεν μπορούν να κληρονομήσουν από κλάσεις. Ο κώδικας

```
interface Student extends Person {}
```

παράγει λάθος μεταγλώττισης.

Οι διεπαφές μπορούν να κληρονομήσουν από απεριόριστο αριθμό διεπαφών. Ο κώδικας

```
public interface Gambler extends CardPlayerI, DicePlayerI {}
```

είναι ορθός.

14.4.2 Η κλάση νικάει

Στην περίπτωση που μία κλάση κληρονομεί τον ίδιο τύπο από κλάση και διεπαφή, τότε κληρονομείται η υλοποίηση της κλάσης και ο κοινός τύπος δεν είναι υποχρεωτικό να επανοριστεί στην παράγωγη κλάση.

```
import java.time.LocalDate;
import java.time.Month;

interface Named {
    public String getfName();
    public String getsName();
}

public class Inherited extends Person implements Named {

    public Inherited(String id, String fName, String sName, LocalDate
birthday) {
```

```
            super(id, fName, sName, birthday);
        }

        public static void main(String[] args) {
            Inherited i=new Inherited("1", "George", "Best",
    LocalDate.of(1950,Month.MARCH, 1));
            System.out.println(i.getfName());
        }
    }
```

Κώδικας 14.9 *Η κλάση Inherited κληρονομεί ίδιους τύπους από τη διεπαφή Named και την κλάση Person*

Η κλάση Inherited είναι παράγωγη της Person. Επομένως κληρονομεί τις μεθόδους της Person ανάμεσα στις οποίες συμπεριλαμβάνονται οι getfName() και getsName(). Παράλληλα, η κλάση υλοποιεί τη διεπαφή Named που ορίζει τις ίδιες μεθόδους. Σε αυτήν την περίπτωση ισχύει ο κανόνας που λέει πως η κλάση νικάει (calss wins), δηλαδή η Inherited κληρονομεί τις υλοποιήσεις της Person. Επομένως, έχει τη δυνατότητα να τις επανορίσει χωρίς όμως να είναι υποχρεωτικό.

14.4.3 Στατικά μέλη

Τα στατικά μέλη δεν κληρονομούνται. Τα στατικά μέλη μπορούν να οριστούν στις διεπαφές με τον ίδιο τρόπο που ορίζονται και στις κλάσεις. Η προσπέλαση σε αυτά γίνεται με προσδιοριστή ονόματος το όνομα της διεπαφής ακολουθούμενο από τον τελεστή τελεία και το όνομα του μέλους.

```
import java.util.Arrays;

public class AllaXaraktiristika {

    public interface Gambler extends CardPlayerI, DicePlayerI {
    }

    public interface Distribution {

        public double[] getDistribution();

        public static Double[] fairDistribution() {
            return new Double[]{1d / 6, 1d / 6, 1d / 6, 1d / 6, 1d / 6, 1d
/ 6};
        }
    }

    public static void main(String[] args) {
        Double[] d = Distribution.fairDistribution();
        System.out.println(Arrays.asList(d));
    }
}
```

Κώδικας 14.10 *Στατικά μέλη στις διεπαφές*

Όπως φαίνεται στον κώδικα 14.10, η διεπαφή Distribution ορίζει το αφηρημένο μέλος getDistribution και το στατικό μέλος fairDistribution. Εφόσον η fairDistribution είναι μία για όλες τις υλοποιήσεις της διεπαφής, λογικό είναι να ορισθεί ως στατική. Η fairDistribution μπορεί να κληθεί ως Distribution. fairDistribution().

14.4.4 Προκαθορισμένες μέθοδοι

Οι προκαθορισμένες μέθοδοι (default methods) στις διεπαφές είναι αποτέλεσμα της πολιτικής της Java να διατηρεί συμβατότητα με όλες τις προηγούμενες εκδόσεις της. Θα το εξηγήσουμε αυτό με το παράδειγμα της διεπαφής Iterator. Αρχικά, η διεπαφή όριζε δύο αφηρημένες μεθόδους, την hasNext() και την next(). Κάποια στιγμή έγινε αντιληπτό πως η πρόσθεση μιας μεθόδου remove() που διαγράφει από την υποκείμενη συλλογή το τρέχον στοιχείο του Iterator θα ήταν χρήσιμη σε πολλές περιπτώσεις. Έως ότου να προστεθεί όμως η remove

είχε αναπτυχθεί παγκοσμίως σημαντικός όγκος κώδικα που χρησιμοποιούσε τον Iterator υλοποιώντας μόνο την hasNext() και την next(). Εάν λοιπόν προστίθετο στην Iterator η remove ως αφηρημένη μέθοδος, ο μεταγλωττιστής θα απαιτούσε την υλοποίησή της με αποτέλεσμα ο προϋπάρχων κώδικας να μην μεταγλωττίζεται. Ωστόσο ο προϋπάρχων κώδικας δεν αξιοποιούσε την remove, καθώς όταν αναπτύχθηκε δεν υπήρχε. Προκειμένου να αντιμετωπιστεί αυτό το πρόβλημα εισήχθησαν στην έκδοση 8 της Java οι προκαθορισμένες μέθοδοι. Οι προκαθορισμένες μέθοδοι παρέχουν υλοποίηση έτσι ώστε οι κώδικες-πελάτες να περνούν μεταγλώττιση χωρίς να απαιτείται να τις υλοποιήσουν, ενώ παράλληλα νέοι κώδικες που τις χρειάζονται μπορούν να τις επανορίσουν.

Σημειώστε πως η λειτουργικότητα των προκαθορισμένων μεθόδων δεν επαρκεί για να χρησιμοποιηθούν παρά μόνο αν επανοριστούν. Πιο συγκεκριμένα, οι προκαθορισμένες μέθοδοι, εφόσον δεν επανοριστούν, απλά παράγουν μια εξαίρεση που πληροφορεί πως η μέθοδος δεν έχει οριστεί.

Τέλος να σημειωθεί πως ο κανόνας η κλάση νικάει ισχύει και για τις προκαθορισμένες μεθόδους.

14.4.5 Ανώνυμες κλάσεις

Μερικές φορές χρειαζόμαστε μια κλάση μόνο μία φορά μέσα στο πρόγραμμά μας. Σε αυτές τις περιπτώσεις, οι διεπαφές μας δίνουν τη δυνατότητα αξιοποίησης ανώνυμων (anonymous) κλάσεων. Αν υποθέσουμε πως χρειαζόμαστε ένα μόνο αντικείμενο τύπου DicePlayerI, τότε μπορούμε να το δημιουργήσουμε όπως δείχνει ο κώδικας 14.11, δηλαδή με τον τελεστή new ακολουθούμενο από τη διεπαφή DicePlayerI και ένα μπλοκ στο οποίο επανορίζουμε τις μεθόδους της διεπαφής και προσθέτουμε οποιαδήποτε άλλα μέλη χρειαζόμαστε. Με αυτόν τον τρόπο έχουμε ένα αντικείμενο με τα χαρακτηριστικά που χρειαζόμαστε, εκεί ακριβώς που το χρειαζόμαστε. Ωστόσο, σε γενικές γραμμές δεν συνίσταται η χρήση μεγάλων και πολύπλοκων ανώνυμων κλάσεων.

```
public class AnonymousDemo {

    public static void main(String[] args) {
        Dice dice = new Dice();
        DicePlayerI jim = new DicePlayerI() {
            @Override
            public int roll(Dice dice) {
                return dice.roll();
            }
        };
        for (int i = 0; i < 50; i++) {
            System.out.println(jim.roll(dice));
        }
    }
}
```

Κώδικας 14.11 *Επίδειξη της ανώνυμης κλάσης*

14.5 Λυμένες Ασκήσεις

14.5.1. Καζίνο

Σε ένα καζίνο υπάρχει η δυνατότητα να παίξει κανείς διαφορετικά παιχνίδια με τράπουλα. Κάποια παιχνίδια απαιτούν και τις 52 κάρτες της τράπουλας και κάποια παίζονται με τράπουλα που περιέχει τις κάρτες από 7 έως τον άσο, δηλαδή με 32 κάρτες. Έτσι το καζίνο χρειάζεται 2 τύπους τράπουλας.
Δημιουργήστε κατάλληλη διεπαφή τράπουλας και δύο κλάσεις που υλοποιούν τη διεπαφή. Η μία κλάση θα αναπαριστά τράπουλα 52 καρτών και η άλλη τράπουλα 32 καρτών. Σε κάθε μία από αυτές τις κλάσεις, υλοποιήστε τον αποθηκευτικό χώρο ως πίνακα από κάρτες. Αναπτύξτε εφαρμογή που αποθηκεύει μια τράπουλα 32 καρτών και μία 52 καρτών σε κατάλληλο πίνακα. Στη συνέχεια για κάθε τράπουλα στον πίνακα, ανακατέψτε την, σηκώστε μια κάρτα, εμφανίστε την κάρτα και στη συνέχεια εμφανίστε την υπόλοιπη τράπουλα. Ελέγξτε την ορθότητα της εξόδου.

Λύση

Καταρχάς, παρουσιάζεται η διεπαφή:

```
public interface Deck {
    public void shuffle();
    public boolean empty();
    public Card pickTop();
}
```

Κώδικας 14.12 *Η διεπαφή Deck*

Οι μέθοδοι shuffle και pickTop απαιτούνται άμεσα από την εκφώνηση. Η μέθοδος empty που ελέγχει αν υπάρχει ή όχι στην τράπουλα κάρτα είναι γενικότερα χρήσιμη. Στη συνέχεια δίνεται η κλάση Deck52 που αναπαριστά τράπουλες των 52 καρτών.

```
import java.util.Random;

public class Deck52 implements Deck {

    final Card[] deck;
    private int cardsInDeck;
    private static final int DECKSIZE = 52;

    public Deck52() {
        deck = new Card[DECKSIZE];
        cardsInDeck = 0;
        for (CardRank r : CardRank.values()) {
            for (Suit s : Suit.values()) {
                deck[cardsInDeck++] = new Card(r, s);
            }
        }
    }

    @Override
    public void shuffle() {
        Random r = new Random();

        for (int i = 0; i < deck.length; i++) {
            int rIdx = r.nextInt(deck.length);
            Card tmp = deck[i];
            deck[i] = deck[rIdx];
            deck[rIdx] = tmp;
        }
    }

    @Override
    public boolean empty() {
        return cardsInDeck == 0;
    }

    @Override
    public Card pickTop() {
        cardsInDeck--;
        return deck[DECKSIZE - cardsInDeck - 1];
    }

    @Override
    public String toString() {
        if (cardsInDeck == 0) {
            return "Empty Deck";
        }
        StringBuilder rVal = new StringBuilder();
```